Veterinary Module Structure, Workflow, and Logic

This document describes the current Pawesome veterinary module after the recent workflow fixes.

1. High-level architecture

text
React veterinary UI
  /veterinary/*
    v apiRequest()
Laravel API
  /api/veterinary/*
    v controllers
Business services
  ServiceBillingService
  VeterinaryInventoryService
  PaymentVerificationService
  ServiceBillingPaymentService
  WorkflowNotifier / NotificationService
    v
MySQL
  appointments
  medical_records
  medical_confinements
  service_item_usages
  service_billing_payment_requests
  payment_settlements
  inventory_items / inventory_batches

The veterinary module connects six related workflows:

1. Appointment workflow
2. Consultation and medical-record workflow
3. Billing workflow
4. Payment verification workflow
5. Inventory usage workflow
6. Medical confinement workflow

Frontend routes are defined in frontend/src/routes/VetRoutes.jsx. Backend veterinary routes are grouped under /api/veterinary in backend/routes/api.php.

2. Frontend structure

All veterinary pages are lazy-loaded and wrapped by ProtectedRoute. The dashboard shell contains the navigation layout, while nested routes render each page.

```
 |  Frontend route  |  Component  |  Purpose  |
 |  /veterinary  |  VetAppointments  |  Default landing page and appointment queue  |
 |  /veterinary/appointments  |  VetAppointments  |  Assigned appointments, filters,
start/cancel/complete actions  |
 |  /veterinary/appointments/:id/consult  |  VetConsultation  |  Consultation form and medical
record editor  |
 |  /veterinary/appointments/:id/billing  |  VetAppointmentBilling  |  Itemized bill and add-on
charges  |
 |  /veterinary/appointments/:id/edit  |  VetEditAppointment  |  Edit appointment while still
editable  |
```

```
 |  /veterinary/medical-confinements  |  VetMedicalConfinements  |  Assigned confinement cases and
progress notes  |
 |  /veterinary/current-boarders  |  VeterinaryCurrentBoarders  |  In-patient/current boarder view
|
 |  /veterinary/history  |  VetHistory  |  Completed/cancelled appointment history  |
 |  /veterinary/customer-profiles  |  VetCustomerProfiles  |  Customers, pets, and medical history
|
 |  /veterinary/reports  |  VetReports  |  Veterinary metrics/reports  |
 |  /veterinary/receipt  |  VetReceipt  |  Receipt display  |
 |  /veterinary/services  |  VetServices  |  Veterinary service reference  |
 |  /veterinary/payroll  |  MyPayroll  |  Shared staff payroll page  |
 |  /veterinary/profile  |  ProfileSettings  |  Shared staff profile page  |
```

Important frontend behavior

VetAppointments.jsx is the operational queue.

- It fetches GET /api/veterinary/appointments.
- It displays assigned appointments only.
- It normalizes backend in_consultation to the frontend display state in_progress.
- needs_confinement remains clickable and can reopen the consultation.
- awaiting_payment shows the consultation as complete and exposes a payment-gated Complete button.
- The list refreshes so receptionist/cashier-side changes appear without a full page reload.

VetConsultation.jsx is the main clinical screen.

- Loads the appointment.
- Finds the medical record for the appointment/pet.
- Supports draft and finalized saves.
- Supports structured prescriptions.
- Supports service add-ons/inventory usage.
- Supports medical confinement recommendation.
- Calls the consultation completion endpoint after finalization.

3. Backend structure

Main controllers

```
 |  Controller  |  Responsibility  |
 |  Veterinary/DashboardController  |  Dashboard data, assigned appointment list, history, reports,
receipt  |
 |  AppointmentController  |  Appointment creation, edit, start, medical fields, completion,
status/cancellation  |
 |  Veterinary/ConsultationWorkflowController  |  Consultation start, consultation completion,
confinement recommendation  |
 |  Veterinary/MedicalRecordController  |  Medical records, prescriptions, vaccinations, inventory
usage  |
 |  MedicalConfinementController  |  Confinement list,
room/admission/care/progress/discharge/release  |
 |  Api/ServiceBillingController  |  Billing summary and charge creation  |
 |  Customer/VeterinaryBillingController  |  Customer-side veterinary invoices and payment requests
 |
 |  Cashier controllers/services  |  Cashier payment queue and verification  |
```

Main services

```
 |  Service  |  Responsibility  |
 |  ServiceBillingService  |  Itemized bill, server-side totals, base service charge, balance,
payment-state sync  |
 |  ServiceBillingPaymentService  |  Customer payment requests for veterinary invoices  |
 |  PaymentVerificationService  |  Cashier proof/cash verification and linked-record
```

```
synchronization  |
 |  VeterinaryInventoryService  |  Stock deduction/restoration and inventory billing items  |
 |  WorkflowNotifier  |  In-app/customer/role notifications  |
 |  PaymentSettlementService  |  Durable settlement ledger used for receipts  |
 |  ReceiptBreakdown  |  VAT-inclusive receipt breakdown and settled/refunded totals  |
```

4. Role and access logic

The veterinary route group uses:

php
auth.api
throttle:api
role:veterinary,vet

Role normalization maps:

text
vet   -> veterinary
veterinarian -> veterinary
super_admin -> admin

admin bypasses normal staff-role restrictions, except customer-only routes.

Data-level scoping

Middleware is not the only protection. Controllers also scope data:

- Regular veterinarians only see appointments where appointments.veterinarian_id equals their user ID.
- Medical records can only be accessed for pets connected to that veterinarian's appointments.
- A veterinarian's confinement list is restricted to medical_confinements.vet_id.
- Admins can see and act on the broader dataset.

5. Database relationships

text
customers
 +--- pets
      +--- appointments
           +--- medical_records
           +--- medical_confinements
                | +--- progress_notes
                | +--- care_logs
           +--- service_item_usages
           +--- service_billing_payment_requests
           +--- payment_settlements

service_requests
 +--- appointments
      +--- billing/payment state is mirrored back when settled

hotel_rooms
 +--- medical_confinements

inventory_items / inventory_batches
 +--- service_item_usages

Important columns:

- appointments.veterinarian_id: assigned vet.
- appointments.service_request_id: linked customer booking/request.
- medical_records.appointment_id + pet_id: consultation record link.
- service_item_usages.service_type = veterinary.
- service_item_usages.service_id = appointment.id.
- medical_confinements.consultation_id = appointment.id.
- medical_confinements.vet_id: assigned veterinarian.

6. Appointment status machine

text
pending
+--- approved
+--- scheduled
+--- cancelled
+--- rejected

approved / scheduled
+--- in_consultation
+--- needs_confinement
+--- treated
+--- awaiting_payment
+--- completed
+--- cancelled

needs_confinement
+--- in_consultation
+--- treated
+--- awaiting_payment
+--- completed
+--- cancelled

treated
+--- awaiting_payment
+--- completed
+--- cancelled

awaiting_payment
+--- completed
+--- cancelled

rejected
+--- pending

in_progress remains a legacy active state. The backend normalizes new starts to in_consultation; the frontend displays both as "Ongoing."

Separate status fields

```
| Field | Values | Meaning |
| appointments.status | pending, approved, in_consultation, needs_confinement,
awaiting_payment, completed, etc. | Service workflow state |
| appointments.payment_status | unpaid, pending, paid, partial, rejected | Financial state |

| medical_records.status | draft, finalized, locked | Clinical record state |
| medical_confinements.status | recommended, admitted, ready_for_discharge, completed, etc. |
In-patient workflow state |
```

Payment being verified does not automatically complete the appointment. A finalized medical record and zero

balance are also required.

7. Normal veterinary workflow

Step 1 - Booking/approval

Before the vet sees the appointment:

1. Customer or staff creates a booking.
2. Receptionist approves/schedules it.
3. Receptionist assigns a veterinarian through veterinarian_id.
4. The appointment appears in the vet's assigned queue.

The vet dashboard list calls:

text
GET /api/veterinary/appointments

It returns assigned appointments in these statuses:

text
approved
scheduled
in_progress
in_consultation
needs_confinement
treated
awaiting_payment

Step 2 - Start appointment

The Start button calls:

text
POST /api/veterinary/appointments/{id}/start

The backend checks:

- Appointment exists.
- Current vet may access it.
- Status is approved or scheduled.
- Optional notes are valid.

Inside one transaction it:

1. Sets status = in_consultation.
2. Sets started_at.
3. Optionally appends start notes.
4. Creates or finds the draft MedicalRecord for (appointment_id, pet_id).

The transaction matters because an appointment should not become active without its consultation record.

There is also a consultation-specific start endpoint:

text
POST /api/veterinary/consultations/{id}/start

That endpoint is more idempotent. It accepts already-active states and does not reset needs_confinement or treated.

Step 3 - Consultation/medical record

On the consult page, the vet can enter:

- Chief complaint
- Symptoms/findings
- Diagnosis
- Treatment plan
- Notes
- Weight
- Temperature
- Heart rate
- Respiratory rate
- Prescriptions
- Vaccinations
- Service add-ons/inventory usage

Draft save calls:

text
PUT /api/veterinary/medical-records/{id}

Final save sends status=finalized.

The record update runs in a transaction covering:

- Main medical_records update
- Prescription replacement
- Vaccination insert/update

Medical-record state

```
 |  Status  |  Meaning  |  Vet edit behavior  |
 |  draft  |  Consultation still being written  |  Creating vet can edit  |
 |  finalized  |  Clinical document is complete  |  Read-only for veterinarians; admin can amend
non-locked records  |
 |  locked  |  Strong clinical lock  |  Nobody can edit  |
 |  deleted  |  Soft delete  |  Only draft records may be deleted  |
```

Step 4 - Optional inventory/service add-ons

Inventory usage uses:

text
POST /api/veterinary/appointments/{id}/inventory-usage

For every submitted item, the service:

1. Validates the requested item and quantity.
2. Locks the inventory_items row.
3. Requires the item to be is_service_consumable.
4. Compares the new quantity with the previous usage for this appointment.
5. Deducts stock when quantity increased.
6. Restores stock when quantity decreased.
7. Uses batch/FEFO deduction.
8. Creates or updates a service_item_usages row.
9. Deletes removed usage rows and restores their stock.
10. Synchronizes billing totals.

All of those writes happen in one transaction. If any item fails validation or stock is insufficient, all stock and

billing changes roll back.

Low-stock notifications are created only after the transaction commits.

Step 5 - Optional confinement recommendation

During an active consultation, the vet can submit:

text
POST /api/veterinary/consultations/{id}/recommend-confinement

Allowed source statuses:

text
in_consultation
in_progress
treated

Required fields:

- diagnosis
- reason_for_confinement
- urgency_level

Optional fields:

- expected stay
- treatment plan
- medication plan
- observation instructions
- special care instructions
- estimated cost

One transaction then:

1. Creates a medical_confinements row with status=recommended.
2. Copies customer/pet/vet/diagnosis details.
3. Updates the appointment to needs_confinement.
4. Copies diagnosis/treatment/medication/reason fields onto the appointment.

After submission:

- Receptionist is notified.
- Customer is notified.
- The frontend marks the recommendation as submitted.
- The appointment can still return to consultation completion.
- The appointment and confinement continue as separate workflows.

Step 6 - Finalize consultation

When the vet clicks Complete Consultation, the frontend:

1. Saves/finalizes the medical record if it is still a draft.
2. Calls:

text
POST /api/veterinary/consultations/{id}/complete

The backend requires:

- Appointment is active: in_consultation, in_progress, needs_confinement, or treated.

- Current vet is assigned, or user is admin.
- A finalized MedicalRecord exists.

Inside one transaction it:

1. Runs ServiceBillingService::finalizeServiceBill.
2. Synchronizes itemized billing totals back onto the appointment.
3. Sets status = awaiting_payment.
4. Stores diagnosis, treatment notes, prescription, and vet remarks.
5. Sets completed_at.

After commit, it notifies:

- Customer: consultation finalized
- Cashier role: bill ready for payment

The key transition is:

text
needs_confinement + finalized medical record
  -> consultation complete
  -> awaiting_payment

Recommending confinement therefore does not trap the appointment.

8. Billing logic

The itemized billing ledger is service_item_usages.

For veterinary:

text
service_type = veterinary
service_id = appointments.id

Billing item types

Type	Meaning
base_service	Original consultation/service price
inventory_usage	Supplies/medicines used during treatment
manual_charge	Staff-entered charge
add_on_service	Additional service charge
discount	Negative/non-billable adjustment

Direct inventory_usage billing entries are rejected; stock-backed usage must go through the veterinary inventory endpoint.

Base service item

ServiceBillingService::ensureBaseServiceItem() guarantees one base billing row using the appointment's authoritative amount:

text
total_amount -> price -> consultation_fee

It locks the appointment row while checking/creating the base item to reduce duplicate creation races.

Billing formulas

syncServicePaymentState() calculates:

text
gross_bill = sum(is_billable items)
discount_total = sum(discount items)
total_bill = max(0, gross_bill - discounts)
total_paid = sum(paid billable items)
balance_due = max(0, total_bill - total_paid)
base_amount = sum(base_service items)
additional_charges = gross_bill - base_amount - discounts

It writes these back onto the appointment:

- base_amount
- additional_charges
- total_amount
- amount_paid
- balance_due
- payment_status
- consultation_fee
- price

Add-charge logic

POST /api/billing/items runs inside a transaction and:

1. Locks the appointment row.
2. Validates the user's role.
3. Ensures the base charge exists.
4. Recomputes total server-side.
5. Rejects invalid discounts.
6. Checks for a recent identical unpaid item to prevent double-submission.
7. Inserts the charge.
8. Synchronizes totals.
9. Marks open veterinary billing payment requests as stale.

9. Payment workflow

There are two relevant payment paths.

A. Original appointment/service-request payment

The original booking payment can be verified through:

- The appointment row
- Its linked service_request

PaymentVerificationService::verifyTablePayment() runs in one transaction and locks the payable row.

For appointments it:

1. Rejects unpayable booking statuses: pending, cancelled, rejected, no_show.
2. Allows payment statuses: pending, unpaid, rejected.
3. Determines whether the collection is cash or digital proof.
4. Applies booking-channel rules.
5. Writes payment_status=paid, paid_at, verified_by, receipt_number, and reference_number.
6. Mirrors the paid state to the linked service_request.
7. Marks the base service billing item paid.
8. Synchronizes the billing ledger.

9. Writes a durable PaymentSettlement.
10. Returns a ReceiptBreakdown.

B. Additional veterinary bill payment request

The customer can request cashier payment or submit GCash/Maya proof through:

text
POST /api/customer/billing/veterinary/{id}/cashier-payment
POST /api/customer/billing/veterinary/{id}/online-payment

This creates a service_billing_payment_requests snapshot when there is an additional unpaid balance.

The request stores:

- Appointment/customer/pet/service metadata
- Full current balance
- Payment channel
- Payment method/reference/proof
- Invoice version hash
- Snapshot of unpaid billable items and discounts

When the cashier verifies it:

1. Appointment and payment request are locked.
2. Invoice version and amount are revalidated.
3. Request is rejected as stale if the bill changed.
4. Snapshot items are locked and marked paid.
5. Billing totals are synchronized.
6. A service_billing_payment settlement is recorded.
7. The payment request becomes paid.
8. The linked service request is updated when the balance reaches zero.

Payment states

text
unpaid
  +--- pending   -> submitted proof/payment request
  +--- paid   -> fully verified and zero balance
  +--- partial   -> some items paid but balance remains
  +--- rejected -> proof rejected

A direct base-payment verification can leave the appointment as partial if add-on items remain unpaid. Completion still requires payment_status=paid and balance_due <= 0.

10. Final appointment completion

The vet's final Complete action calls:

text
POST /api/veterinary/appointments/{id}/complete

The backend requires all of these:

1. Appointment status is completable.
2. Finalized medical record exists.
3. payment_status === paid.
4. Billing balance is zero.

Only then does it set:

text
status = completed
completed_at = now()

This keeps the order strict:

text
clinical completion
  -> billing ready
  -> payment verified
  -> appointment completed

Payment alone does not complete the appointment.

11. Confinement workflow

The appointment's confinement recommendation creates a separate medical_confinements workflow.

Confinement statuses

text
recommended
 -> approved_for_admission
 -> admitted
 -> under_observation
 -> under_treatment
 -> ready_for_discharge
 -> completed

Other accepted model statuses:

text
discharged
cancelled

The current release endpoint writes completed, while discharged remains an accepted historical/model status.

Step-by-step confinement flow

1. Vet recommends

text
POST /api/veterinary/consultations/{id}/recommend-confinement

Creates:

text
medical_confinements.status = recommended
appointments.status = needs_confinement

2. Receptionist assigns room

text
POST /api/receptionist/medical-confinements/{id}/assign-room

Checks:

- Status is recommended or approved_for_admission
- Room exists
- Room is available unless it is already the assigned room

Sets:

text
room_id
estimated_cost
status = approved_for_admission

3. Receptionist admits pet

text
POST /api/receptionist/medical-confinements/{id}/admit

Requires a room assignment.

In one transaction:

- Confinement becomes admitted
- admitted_by and admitted_at are set
- Room becomes occupied

4. Vet records treatment

Veterinary routes can add progress notes and move the patient into:

text
under_observation
under_treatment

Progress-note creation and any optional status update are committed together.

Receptionist-side care logs can also be added while the animal is admitted.

5. Vet clears discharge

text
POST /api/veterinary/medical-confinements/{id}/clear-for-discharge

Allowed only from:

text
admitted
under_observation
under_treatment

One transaction:

- Sets status=ready_for_discharge
- Records who cleared it and when
- Creates a discharge_clearance progress note

6. Confinement payment

Confinement has its own payment state, separate from the appointment:

text
unpaid -> pending -> paid
-> rejected

The customer may submit GCash/Maya proof. Cashier verification uses the payment type medical_confinement.

A recommended confinement is not payment-verifiable because it has not reached the operational admission flow.

7. Receptionist releases pet

text
POST /api/receptionist/medical-confinements/{id}/release

Requires:

- status=ready_for_discharge
- payment_status=paid

In one transaction:

- Confinement becomes completed
- discharged_by and discharged_at are set
- Room becomes available

12. Transaction and concurrency behavior

```
| Operation | Transaction contents | Locking |
| Appointment start | Status update + medical record firstOrCreate | No explicit appointment
lock |
| Consultation start | Status/vet assignment + medical record firstOrCreate | No explicit
appointment lock |
| Consultation complete | Billing sync + appointment status/clinical fields | Nested
billing/base-item transaction |
| Confinement recommendation | Confinement insert + appointment update | No explicit
appointment lock |
| Medical record update | Record update + prescription/vaccination changes | DB transaction
|
| Billing item add | Base item + charge + totals + stale payment requests | Appointment
lockForUpdate |
| Inventory usage | Stock changes + usage rows + billing sync | Inventory row lockForUpdate
|
| Payment verification | Payment state + linked service request + billing + settlement |
Payable row lockForUpdate |
| Billing payment verification | Payment request + appointment + item updates + settlement |
Appointment/request/item locks |
| Confinement admission | Confinement status + room occupancy | DB transaction |
| Progress note/status | Note insert + optional confinement status | DB transaction |
| Discharge clearance | Status + clearance note | DB transaction |
| Release | Confinement status + room availability | DB transaction |
| Appointment cancellation | Appointment status + linked service-request status | DB
transaction |
```

Practical meaning

A transaction means:

text
success -> all related writes commit together
failure -> every write inside the transaction rolls back

For example, if billing synchronization fails during consultation completion, the appointment does not remain incorrectly moved to awaiting_payment.

lockForUpdate() means concurrent operations wait for the current transaction before reading/updating the same row. This is especially important for billing totals, inventory stock, payment verification, stale payment-request checks, and linked service-request mirroring.

13. Receipt logic

GET /api/veterinary/receipt/{id}:

1. Loads the assigned appointment.
2. Calls the billing service to get itemized billing.
3. Finds paid settlements tied to appointment and service_billing_payment.
4. Falls back to the appointment's paid fields for legacy rows without settlement records.
5. Calls ReceiptBreakdown::build().
6. Returns VAT, subtotal, total, paid amount, refunds/voids, balance, itemized services, and receipt number.

ReceiptBreakdown treats prices as VAT-inclusive and calculates the net/VAT split. The settlement ledger is authoritative for what was actually paid.

14. Notification and audit behavior

Notifications happen at important workflow points:

- Consultation started
- Consultation finalized
- Cashier billing notification
- Confinement recommended
- Room assigned
- Pet admitted
- Progress updated
- Ready for discharge
- Pet released
- Payment verified/rejected
- Low inventory

Some notifications are WorkflowNotifier calls after transactions. Payment-related notification records are intentionally inside the payment transaction so they roll back if verification fails.

ActivityLog records significant staff actions:

- Appointment started
- Medical information updated
- Appointment completed
- Status changes
- Confinement admitted
- Confinement cleared
- Confinement released
- Payment verified/rejected

15. Current end-to-end sequence

text
Customer/receptionist creates booking
v
Receptionist approves/schedules and assigns vet

```
        v
Vet sees appointment
        v
POST appointments/{id}/start
  - status: in_consultation
  - draft medical_record created
        v
Vet fills record, prescriptions, vitals, inventory usage
        v
Optional confinement recommendation
  - medical_confinements: recommended
  - appointment: needs_confinement
        v
Vet finalizes medical_record
        v
POST consultations/{id}/complete
  - billing synced
  - appointment: awaiting_payment
        v
Customer submits cashier/online payment request
        v
Cashier verifies payment
  - items marked paid
  - payment settlement recorded
  - appointment payment synchronized
        v
Vet clicks Complete
  - finalized record required
  - payment_status=paid required
  - balance_due=0 required
        v
appointment.status = completed
```

The parallel confinement branch continues independently:

```
text
recommended
  -> room assigned
  -> admitted
  -> observation/treatment
  -> ready_for_discharge
  -> payment verified
  -> released/completed
```

16. Important implementation details

- needs_confinement means "a confinement recommendation exists"; it does not mean the appointment itself is over.
- ready_for_discharge is the canonical backend confinement status. The frontend uses that value.
- finalized is the normal completed medical-record state; locked is a stronger legal/audit-style lock.
- payment_status=paid alone is not enough to complete an appointment; the itemized balance must also be zero.
- Customer billing payment requests use an invoice_version hash. If the bill changes, the old request becomes stale.

- The appointment list endpoint includes in_consultation and needs_confinement. The dashboard summary endpoint uses a narrower active-status list, so its count cards can undercount those states even though the appointment queue itself displays them correctly.